

Mixing Colors: Exploring Digital Images through the Bayer Filter

Brittany Morago
University of North Carolina Wilmington
Wilmington, USA
moragob@uncw.edu

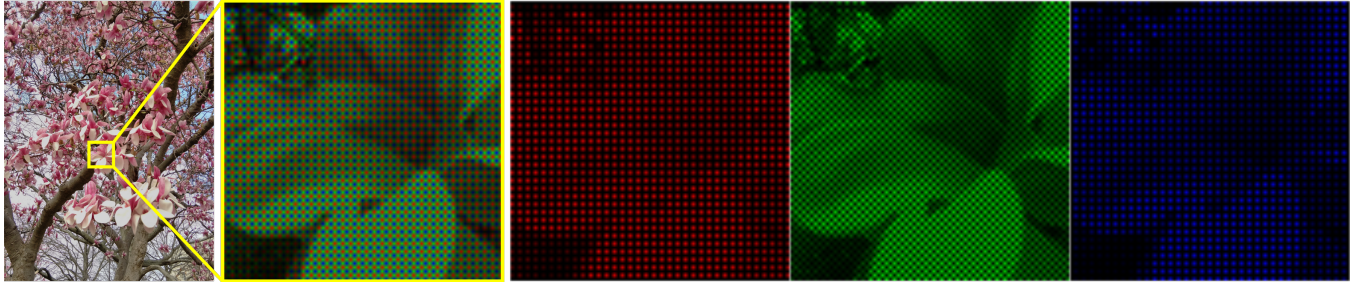


Figure 1: Creating a Bayer Filter version of an image. Left to right: Original image, zoomed in view of the full Bayer Filter version, and red, green, and blue color channels copied over individually.

Abstract

The Bayer Filter assignment is designed for the beginning of an undergraduate Computer Graphics class. It covers digital image formation, input, and output, iterating over a 3-D array, and basic C++ syntax all in a visually interesting context. Students work through the steps of the Bayer Filter algorithm and equations on a worksheet and then use all their written math to guide their coding process. The assignment is structured so that each segment of the pipeline can be easily visualized and debugged.

CCS Concepts

• **Applied computing** → **Interactive learning environments**; • **Computing methodologies** → **Image processing**.

Keywords

Interactive assignments, computer graphics, digital image processing

ACM Reference Format:

Brittany Morago. 2026. Mixing Colors: Exploring Digital Images through the Bayer Filter. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educator's Forum '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3799829.3812519>

1 Introduction

A standard topic in undergraduate Computer Graphics courses is digital image input, output, and formation. It often includes the typical data structure of images used in image processing libraries - a grid of pixels with the origin in top left, 3 numbers stored at

each pixel to represent RGB values, and how different mixes of red, green, and blue can recreate a decent portion of the visible spectrum. To work with this data structure, students need to be able to iterate over a 3-D grid. Often times, students are drawn into graphics because of its visual nature and applications to gaming and entertainment. They have not always mastered utilizing nested loops and are still working on conceptualizing using code to access data in anything more complex than a 1-D array. Graphics courses may also come with learning a new programming language such as C++ on top of learning CG theories, math, and algorithms.

This assignment uses a deep dive into the Bayer Filter [Bayer 1976] to meet these learning outcomes and is given at the beginning of a CG course to prepare for subsequent assignments. Students are asked to write a program that mimics the full pipeline of a digital camera sampling light measurements of a scene (Fig. 1) and reconstructing a complete RGB image. The idea is to treat an input image as the "real-world" data, and measure only the red, green, or blue light at each pixel in the same pattern a camera sensor guided by a Bayer Filter would use. Students reconstruct a full RGB image using only the sampled Bayer Filter measurements by copying over whatever color channel is already stored at the pixel and averaging values from the 8 neighboring pixels to approximate the other 2 color channels. Students observe the quality loss between the "original" image and the "reconstructed" image and discuss how this level of data loss is acceptable when images are captured at a high enough resolution.

By running different sections of their code individually, students can see first hand how red, green, and blue light can be mixed in different proportions to create a wide gamut of colors. They have several opportunities to practice stepping around a grid of pixels in different patterns and master the syntax for accessing and updating the values stored at different pixels. They learn C++ syntax for basic programming structures to use in future graphics assignments. They also work with a program that has discrete sections where different sets of nested loops are used to address



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Educator's Forum '26, Los Angeles, CA, USA*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2552-4/26/07
<https://doi.org/10.1145/3799829.3812519>

Summary	Study digital image formation by creating a Bayer filter pipeline.
Learning Outcomes	Ability to iterate over 3D grid (image) and learn basic C++ syntax
Classification(s)	Fundamentals, Image Manipulation
Audience	Computer Science and Digital Arts juniors
Dependencies	2 semesters of programming (any language)
Prerequisites	N/A (first assignment in CG course)
Strengths	Link math with color and images to create fun effects
Weaknesses	Understanding how to access individual pixels is challenging
Variants	Handle edge cases during reconstruction and consolidate loops
Assessment	Points assigned to each color channel's section in the Bayer and the Reconstruction portions and graded on completion

Table 1: Bayer Assignment Metadata

one portion of this process at a time. Students get to practice their debugging skills by breaking a program down into tiny pieces and visualizing what is happening at each stage.

2 Demo Code

Students are given demo code for filling in the green pixels to 1) show how to load and display an image, 2) cover C++ syntax for loops, and 3) provide syntax for accessing values stored at color channel of each pixel (Fig. 1, 4th image). With this they have all the tools they need to complete the assignment. It becomes purely a logic exercise.

3 Pre-Programming Worksheet

Before diving into the coding portion of this assignment, students complete a written worksheet where they determine how to step through the digital image data structure to visit only one color channel at a time according to the Bayer pattern. They also write down which neighbors need to be averaged together to reconstruct each color channel.

Over several years of giving this assignment, I observed students often diving straight into coding, getting unexpected errors, and then just trying different numbers in the loops and averaging formulas almost like guess work, hoping everything would magically start working. This worksheet starts with a grid of the Bayer Filter and is meant to provide the structure and support for students to break their assignment down into bite-size pieces and work on a single problem at a time.

In preparation for programming creating a Bayer Filter version of an input image, students answer the questions:

- At what coordinate in the top left is the first blue pixel?
- What size step do you need to take to get to the next column with a blue pixel?
- What size step do you need to take to get to the next row with a blue pixel?

and repeat this for red and green. The values students write on their worksheets are the exact values they put into the for loops

for where to start and how to increment. (Green's information is provided as part of the demo code.)

Then for the reconstruction portion, students answer the questions:

- At what coordinate in the top left is the first blue pixel that has neighbors on all sides? *Skip the top row and left column.*
- What size step do you need to take to get to the next column with a blue pixel?
- What size step do you need to take to get to the next row with a blue pixel?
- For any pixel at coordinate (x,y) that has a blue measurement in the Bayer filter image:
 - What pixel coordinates will you use to determine (x,y)'s red color in the reconstructed image?
 - What pixel coordinates will you use to determine (x,y)'s green color in the reconstructed image?
 - What pixel coordinates will you use to determine (x,y)'s blue color in the reconstructed image?

These questions guide students in how to initialize their for loops and also tell them what relative pixel coordinates to use in their averaging formulas. There are 4 sets of these questions on the reconstruction portion to correspond to the averaging formulas needed for blue locations in the Bayer filter, red locations, green locations with blues above/below and reds on the sides, and green locations with blues on the sides and reds above/below.

4 Bayer Filter Pipeline Implementation

Once the worksheet is completed, students are asked to add onto the provided demo code to 1) create a Bayer Filter version of the image and 2) reconstruct a version of the image using only the Bayer Filter measurements.

The Bayer Filter part simply requires copying over the correct pixel at each location. Students are asked to complete this in 3 sections, working with just one color channel at a time. A set of nested loops for copying green, another for red, and another for blue. They should run their code after each one and zoom in to see the Bayer pattern being constructed.

The reconstruction portion deals with averaging neighboring pixels in the correct configuration. Students are asked (at least initially) to write 4 sets of nested loops to handle each of these scenarios separately. For instance, when reconstructing pixels that have a blue measurement in the Bayer Filter, they will need to implement Equations 1-3 for copying over and averaging pixel values.

$$G = \frac{G_1 + G_2 + G_3 + G_4}{4.0} \quad (1)$$

$$R = \frac{R_1 + R_2 + R_3 + R_4}{4.0} \quad (2)$$

$$B = B_{bayer} \quad (3)$$

G_1 through G_4 are the neighbors above, below, and on the sides and R_1 through R_4 are the neighbors on the diagonal. When reconstructing green measurement pixels, only 2 neighbors will be average for the blue and red channels. Students have the option of ignoring the borders during reconstruction. On high resolution images, skipping two rows and two columns does not have a noticeable impact on the visual result.

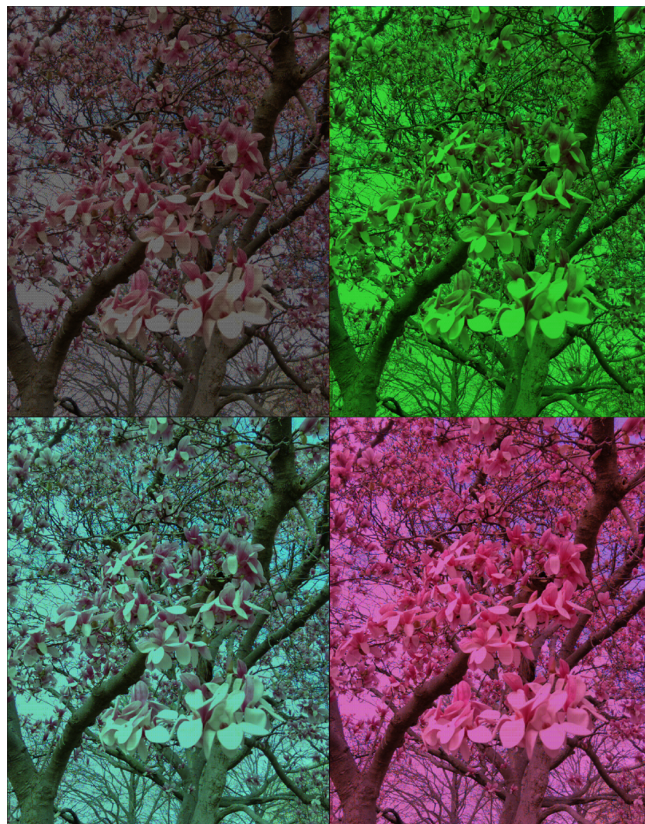


Figure 2: Common errors encountered with implementing reconstruction stage that come with skipping pixels (top left) or copying over the wrong color channels (other 3). These errors do lead to some interesting color mixing.

Again, many students this assignment is designed for are still building their programming skills and working with C++ for the first time. Handling each possible case with a separate set of nested loops provides practice in isolating, visualizing, and debugging each step of their programs. Figure 2 shows visual representations of some of the common errors students run into with this assignment. While these are incorrect results, they are interesting to look at and tend to keep students engaged through the debugging process.

In our program students are generally moving from Python to C++ and encounter integer division and automatic rounding for the first time. This assignment also teaches them to make decisions about ints vs. floats in their code when implementing Equations 1-3.

5 Bonus

Students who are looking for an extra logic/programming challenge have two options with the Bayer assignment. One modification they can make is to combine all the steps for creating the Bayer filter into a single set of nested loops and for doing reconstruction into a single set of nested loops. Each of these sets needs to hit every pixel and either choose the correct combination of formulas on a case by case basis or even possibly adapt the averaging formulas

based on what color channels amongst the neighbors have stored values.

The second bonus option is to handle edge and corner cases in the reconstruction stage. Students have to adapt the RGB averaging formulas to handle the different scenarios they may encounter along edges. For instance, a red pixel will only have 2 blue neighbors instead of 4. The code will have to handle accessing the 2 correct locations and update the averaging formula to divide by the correct number of found neighbors. For corners, the programmer will always know what color channel was measured for the top left corner based on how they arranged their code. However, the other 3 corners could have any color channel stored based on different sized images. This bonus implementation has to determine what color channel the grid stops on in each direction.

6 Conclusion

Students often report having fun with this assignment as they learn how to play with images through code and get interesting results by dissecting the different color channels. Students with stronger programming skills enjoy the added puzzle of handling edge cases.

Working out the steps of this assignment on paper can seem trivially easy. However, students attempting to work out this logic in their heads while learning to work with images and C++ for the first time can lead to a lot of time consuming bugs. By adding in the pre-worksheet portion, I have seen a huge jump in 1) the rate students are able to quickly and correctly implement the Bayer Filter portion of the assignment and 2) the number of students tackling the Bonus portions of this assignment as they have more time and confidence to do so. I have also had a decrease in the percentage of my students redoing this assignment. Before I incorporated the pre-programming worksheet into this assignment, I would have as high as a third of the class redo this assignment because of the number of programming errors. After adding this step, I am down to less than 10% of a class each semester needing to reapproach this assignment. Whereas before, a large chunk of my classes would take a few days to get this done, getting hung up on bugs in their loops, most of class will complete this part in a few minutes during lecture. They also get through the reconstruction stage more directly and wrap-up this assignment with more confidence and enthusiasm for the rest of their semester in Computer Graphics.

Acknowledgments

Thank you to my UNCW CSC 370 Computer Graphics students.

References

Bryce Bayer. 1976. Color imaging array. *United States Patent, no. 3971065* (1976).